

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-a54e60d6bb60df878.jsonl`  
- SHA-256: `cb096897c4295ff8291c6cd35fb56741cca1f5ea17e3b77f73c624e334b63420`  
- Source modified: `2026-06-18T10:29:13+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_1d39e81c-851`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T10:20:58.686Z)

You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):

1. SETUP: git fetch origin && git checkout -B <BRANCH> origin/master (base MUST be origin/master).
Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you will touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix" anything).
Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the test (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by changing production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to logic, outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:

- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure move), STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never -A/./-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):

```
python run_all_tests.py  
python -m ruff check .  
python -m mypy backend training  
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70  
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):  
git add <explicit files> ; git commit -m "<conventional msg>" # last line: Co-Authored-By:  
Claude Opus 4.8 <noreply@anthropic.com>  
git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "... " --body  
"... " # NO merge
```

ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened' | 'held' | 'gate-failed' | 'error'),
pr_url ("" + reason if none), char_tests (what you added + that they passed pre-refactor),
diffstat,
suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR ITEM: db-execute-write-dedup (branch refactor/db-execute-write-helper) ===
ITEM: backend/infrastructure/database.py has ~14 bool-returning single-statement write methods
that
each repeat connect/commit/except-logger.error(...)/return False boilerplate with a PER-METHOD
log message.
Introduce a private `\_execute\_write(self, sql, params, error\_msg) -> bool` and route ONLY the
bool-returning
SINGLE-STATEMENT writers through it, PRESERVING each method's exact log message text (pass it as
error_msg).
EXCLUDE: lastrowid-returning writers
(add_play_segment/add_candidate_segment/upsert_user_model/claim_due_job)
and multi-statement ones (e.g. prune_segments) ? leave them untouched.
CHAR TESTS FIRST: failure (return-False) branches are partially covered. Add tests that force a
write to
raise sqlite3.Error (e.g. monkeypatch the connection / pass a bad value) and assert the method
returns False
AND logs its specific message ? for at least 2 of the routed methods ? passing on UNCHANGED code
(the exact
per-method message must be asserted so the dedup can't silently change it). THEN dedup. If a
reviewer would
see ANY log-message change, you got it wrong.

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-3 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T10:21:01.203Z)

I'll start by setting up the worktree on the correct base branch, then write characterization tests before refactoring.

3. user (2026-06-18T10:21:05.400Z)

branch 'refactor/db-execute-write-helper' set up to track 'origin/master'.
Switched to a new branch 'refactor/db-execute-write-helper'

```
---HEAD---
e67add172f5b620025f31489fde2fb9ff45465ac
---ORIGIN/MASTER---
e67add172f5b620025f31489fde2fb9ff45465ac
---BRANCH---
refactor/db-execute-write-helper
```

4. assistant (2026-06-18T10:21:07.281Z)

HEAD matches origin/master and the branch is correct. Now let me read the target file.

5. user (2026-06-18T10:21:09.033Z)

```
1 import sqlite3
2 import json
3 import logging
4 from contextlib import contextmanager
5 from typing import List, Dict, Any, Optional, Tuple
6
```

```

7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         @staticmethod
46         def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
47             """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runably.
48
49             A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
50             transaction, so an interrupted v3/v4 block can leave the column added while
51             ``user_version`` stays un-bumped ? and the next open would re-run the same
52             ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
53             future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
54             Swallow
55             ONLY that specific error so the ladder stays crash-safe, matching the re-
56             runnable
57             ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
58             """
59             try:
60                 cursor.execute(ddl)
61             except sqlite3.OperationalError as exc:
62                 if "duplicate column name" not in str(exc).lower():
63                     raise
64
65         def _init_db(self):
66             """Initializes tables for videos, play_segments, and events."""
67             with self._connect() as conn:
68                 cursor = conn.cursor()
69
70                 self._create_base_tables(cursor)

```

```

69         # Versioned migrations live here. PRAGMA user_version is the schema
70         # contract ? bump it for every change and add an ordered `if version < N`
71         # block below. Tests assert the expected version, so accidental drift fails
72
73         version = cursor.execute("PRAGMA user_version").fetchone()[0]
74
75         if version < 1:
76             self._migrate_to_v1(cursor)
77
78         if version < 2:
79             self._migrate_to_v2(cursor)
80
81         if version < 3:
82             self._migrate_to_v3(cursor)
83
84         if version < 4:
85             self._migrate_to_v4(cursor)
86
87         if version < 5:
88             self._migrate_to_v5(cursor)
89         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
90 user_version = 6
91
92         conn.commit()
93
94     def _create_base_tables(self, cursor):
95         """Create the videos / play_segments / events base tables (idempotent)."""
96         # Videos Table
97         cursor.execute("""
98             CREATE TABLE IF NOT EXISTS videos (
99                 id TEXT PRIMARY KEY,
100                 filepath TEXT NOT NULL,
101                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
102                 status TEXT NOT NULL,
103                 validation_results TEXT
104             )
105         """)
106
107         # Play Segments Table (Extensible metadata JSON)
108         cursor.execute("""
109             CREATE TABLE IF NOT EXISTS play_segments (
110                 id INTEGER PRIMARY KEY AUTOINCREMENT,
111                 video_id TEXT NOT NULL,
112                 start_time REAL NOT NULL,
113                 end_time REAL NOT NULL,
114                 duration REAL NOT NULL,
115                 server TEXT,
116                 receiver TEXT,
117                 metadata TEXT,
118                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
119             )
120         """)
121
122         # Events Table
123         cursor.execute("""
124             CREATE TABLE IF NOT EXISTS events (
125                 id INTEGER PRIMARY KEY AUTOINCREMENT,
126                 segment_id INTEGER NOT NULL,
127                 timestamp REAL NOT NULL,
128                 type TEXT NOT NULL,
129                 player TEXT,
130                 details TEXT,
131                 FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE

```

```

132
133 def _migrate_to_v1(self, cursor):
134     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
135     # Common fields are columns; detector-specific metrics go in `stats` JSON,
136     # so a new segmenter reuses this table by setting its own `detector`/`stats`.
137     cursor.execute("""
138         CREATE TABLE IF NOT EXISTS candidate_segments (
139             id INTEGER PRIMARY KEY AUTOINCREMENT,
140             video_id TEXT NOT NULL,
141             detector TEXT NOT NULL,
142             chunk_index INTEGER,
143             start_time REAL NOT NULL,
144             end_time REAL NOT NULL,
145             duration REAL NOT NULL,
146             confidence REAL,
147             stats TEXT,
148             detection_params TEXT,
149             confirmed_segment_id INTEGER,
150             human_label TEXT,
151             notes TEXT,
152             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
153             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
154             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
155         )
156     """)
157     cursor.execute("""
158         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
159             ON candidate_segments(video_id, detector)
160     """)
161     cursor.execute("PRAGMA user_version = 1")
162
163 def _migrate_to_v2(self, cursor):
164     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
165     # /api/process request. `status` is the EXECUTION state of the job
166     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
167     # that failed validation) lives inside `result` JSON as result["status"].
168     # `result` is the full sync-era response dict (incl. report paths), so the
169     # observability report is the job's artifact, per the plan.
170     cursor.execute("""
171         CREATE TABLE IF NOT EXISTS jobs (
172             id TEXT PRIMARY KEY,
173             video_path TEXT NOT NULL,
174             video_id TEXT,
175             segmenter TEXT,
176             player_pool TEXT,
177             max_frames INTEGER,
178             status TEXT NOT NULL DEFAULT 'queued',
179             progress TEXT,
180             error TEXT,
181             result TEXT,
182             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
183             started_at TIMESTAMP,
184             finished_at TIMESTAMP
185         )
186     """)
187     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
188     cursor.execute("PRAGMA user_version = 2")
189
190 def _migrate_to_v3(self, cursor):
191     # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
192     # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
193     # due
194     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
195     # re-claims it via `claim_due_job` when due ? with NO master node: the table IS

```

```

195         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
196         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
197         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
198         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
199         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
200         cursor.execute("PRAGMA user_version = 3")
201
202     def _migrate_to_v4(self, cursor):
203         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
204         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
205         # path can scope rows to an owner. **NULL = local install = byte-identical to
206         # today** ? every existing row stays NULL and every existing query (which never
207         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
208         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
them.
209         # Additive ONLY: no served-behavior change rides on this slice.
210         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
211         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
212         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
213         # of index churn while making the eventual per-user lookups cheap.
214         cursor.execute(
215             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
216             "WHERE user_id IS NOT NULL")
217         cursor.execute(
218             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
219             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
220         cursor.execute("PRAGMA user_version = 4")
221
222     def _migrate_to_v5(self, cursor):
223         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
224         # version): the resolved per-user `fusion_config` overlay + an INLINE
225         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
226         # and provenance that earned it. `is_active` pins the one row the serving bridge
227         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
228         # only persists/loads it ? NOTHING reads it on the served path yet.
229         #
230         #   user_id           ? owner of the model (NULL allowed = the local install)
231         #   version           ? monotonic per-user model version
232         #   baseline_version  ? the shared baseline this was fit against (upgrade
switch)
233         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
234         #   fusion_config     ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
235         #   classifier_json   ? JSON LogisticRegression.to_dict() (None = config-only
model)
236         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
237         #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n trust floor
input)
238         #   is_active         ? the one resolved row for this user (partial-unique
below)
239         cursor.execute("""
240             CREATE TABLE IF NOT EXISTS user_models (
241                 id INTEGER PRIMARY KEY AUTOINCREMENT,
242                 user_id TEXT,
243                 version INTEGER NOT NULL DEFAULT 1,
244                 baseline_version TEXT,

```

```

245         feature_schema_hash TEXT,
246         fusion_config TEXT,
247         classifier_json TEXT,
248         promotion_evidence TEXT,
249         n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
250         is_active INTEGER NOT NULL DEFAULT 0,
251         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
252     )
253     """
254     # One model version per user (NULLs compare distinct in SQLite, so the local
255     # install can still carry multiple versioned rows; the active-row guard below
256     # is what enforces a single served model).
257     cursor.execute("""
258         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
259         ON user_models(user_id, version)
260     """)
261     # At most ONE active model per user: a partial-unique index over the active
rows.
262     # (SQLite treats NULL user_id rows as distinct, so the local install's single
263     # active row is still guarded ? there is one local user.)
264     cursor.execute("""
265         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
266         ON user_models(user_id) WHERE is_active = 1
267     """)
268     cursor.execute("PRAGMA user_version = 5")
269
270     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
271         """Register or update a video row WITHOUT touching its child segments.
272
273         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
274         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
275         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
276         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
277         row in place; clearing segments is now an explicit, deliberate operation
278         (clear_video_data / prune_segments).
279         """
280         try:
281             with self._connect() as conn:
282                 conn.cursor().execute(
283                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
284                     "ON CONFLICT(id) DO UPDATE SET "
285                     "filepath=excluded.filepath, status=excluded.status, "
286                     "validation_results=excluded.validation_results",
287                     (video_id, filepath, status, json.dumps(validation_results))
288                 )
289                 conn.commit()
290             return True
291         except Exception as e:
292             logger.error(f"Failed to add video: {e}")
293             return False
294
295     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
296         """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
297
298         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
299         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
300         """
301         with self._connect() as conn:
302             cur = conn.cursor()

```

```

303         p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
304                             (video_id,)).fetchone()[0]
305         c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
306                             (video_id,)).fetchone()[0]
307         return int(p), int(c)
308
309     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
310                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
311                       cand_after: Optional[int] = None) -> None:
312         """Delete play/candidate segments by id range (events cascade from
play_segments).
313
314         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
315         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
316         and keep the OLD ones when the run failed/produced nothing (`*_after`).
317         """
318         with self._connect() as conn:
319             cur = conn.cursor()
320             if play_upto is not None:
321                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
322             if play_after is not None:
323                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
324             if cand_upto is not None:
325                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
326             if cand_after is not None:
327                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
328             conn.commit()
329
330     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
331         try:
332             with self._connect() as conn:
333                 cursor = conn.cursor()
334                 cursor.execute(
335                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
336                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
337                 )
338                 conn.commit()
339                 return cursor.lastrowid
340         except Exception as e:
341             logger.error(f"Failed to add play segment: {e}")
342             return None
343
344     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
345         try:
346             with self._connect() as conn:
347                 conn.cursor().execute(
348                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
349                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
350                 )
351                 conn.commit()

```



```

352         return True
353     except Exception as e:
354         logger.error(f"Failed to add event: {e}")
355         return False
356
357     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
358                               end_time: float,
359                               chunk_index: Optional[int] = None, confidence:
360                               Optional[float] = None,
361                               stats: Optional[Dict[str, Any]] = None,
362                               detection_params: Optional[Dict[str, Any]] = None) ->
363                               Optional[int]:
364         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
365         detector-specific dicts stored as JSON. Returns the new candidate id, or
366         None."""
367         try:
368             with self._connect() as conn:
369                 cursor = conn.cursor()
370                 cursor.execute(
371                     """INSERT INTO candidate_segments
372                     (video_id, detector, chunk_index, start_time, end_time, duration,
373                     confidence, stats, detection_params)
374                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
375                     (video_id, detector, chunk_index, start_time, end_time, end_time -
376                     start_time,
377                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
378                     {})))
379                 conn.commit()
380                 return cursor.lastrowid
381         except Exception as e:
382             logger.error(f"Failed to add candidate segment: {e}")
383             return None
384
385     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
386         """Record that a candidate detection was confirmed as a given play_segment."""
387         try:
388             with self._connect() as conn:
389                 conn.cursor().execute(
390                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
391                     ?",
392                     (segment_id, candidate_id)
393                 )
394                 conn.commit()
395             return True
396         except Exception as e:
397             logger.error(f"Failed to link candidate {candidate_id} to segment
398             {segment_id}: {e}")
399             return False
400
401     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
402                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
403         """Fetch candidate detections for the annotation / fine-tuning workflow.
404         `stats` and `detection_params` are deserialized from JSON."""
405         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
406         params: List[Any] = [video_id]
407
408         if detector:
409             query += " AND detector = ?"
410             params.append(detector)
411
412         if only_unlabeled:
413             query += " AND human_label IS NULL"
414
415         query += " ORDER BY start_time"

```

```

409         candidates = []
410         with self._connect() as conn:
411             rows = conn.cursor().execute(query, params).fetchall()
412             for row in rows:
413                 d = dict(row)
414                 d["stats"] = json.loads(d["stats"] or "{}")
415                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
416                 candidates.append(d)
417         return candidates
418
419     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
420         with self._connect() as conn:
421             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
422 (video_id,)).fetchone()
423             if row:
424                 d = dict(row)
425                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
426                 return d
427             return None
428
429     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
430         """
431         Dynamically queries play segments based on filters:
432         - duration_min: float
433         - server: string
434         - receiver: string
435         - event_type: string (e.g. smash, foul)
436         """
437         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
438         params = [video_id]
439
440         if filters.get("duration_min"):
441             query += " AND r.duration >= ?"
442             params.append(filters["duration_min"])
443
444         if filters.get("server"):
445             query += " AND r.server = ?"
446             params.append(filters["server"])
447
448         if filters.get("receiver"):
449             query += " AND r.receiver = ?"
450             params.append(filters["receiver"])
451
452         if filters.get("event_type"):
453             # Check if there is an event of a specific type inside this segment
454             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
455             params.append(filters["event_type"])
456
457         segments_list = []
458         with self._connect() as conn:
459             cursor = conn.cursor()
460             rows = cursor.execute(query, params).fetchall()
461
462             for row in rows:
463                 r_dict = dict(row)
464                 r_id = r_dict["id"]
465                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
466
467                 # Fetch associated events
468                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
469 (r_id,)).fetchall()
470                 r_dict["events"] = []
471                 for er in event_rows:

```

```

470         e_dict = dict(er)
471         e_dict["details"] = json.loads(e_dict["details"] or "{}")
472         r_dict["events"].append(e_dict)
473
474         segments_list.append(r_dict)
475
476     return segments_list
477
478     def clear_video_data(self, video_id: str):
479         with self._connect() as conn:
480             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
481             conn.commit()
482
483     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
484     # Writers follow the repo convention (swallow + logger.error, return bool); the
485     # worker logs loudly on a failed transition so a job can't silently wedge.
486
487     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
488                   player_pool: Optional[List[str]] = None,
489                   max_frames: Optional[int] = None,
490                   policy: Optional[Dict[str, Any]] = None) -> bool:
491         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
492         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
493         try:
494             with self._connect() as conn:
495                 conn.cursor().execute(
496                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
497                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
498                     (job_id, video_path, segmenter,
499                      json.dumps(player_pool) if player_pool is not None else None,
500                      max_frames, json.dumps(policy) if policy is not None else None)
501                 )
502                 conn.commit()
503             return True
504         except Exception as e:
505             logger.error(f"Failed to create job {job_id}: {e}")
506             return False
507
508     def mark_job_running(self, job_id: str) -> bool:
509         try:
510             with self._connect() as conn:
511                 conn.cursor().execute(
512                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
513                     "progress='starting' WHERE id = ?", (job_id,))
514                 conn.commit()
515             return True
516         except Exception as e:
517             logger.error(f"Failed to mark job {job_id} running: {e}")
518             return False
519
520     def set_job_progress(self, job_id: str, progress: str) -> bool:
521         try:
522             with self._connect() as conn:
523                 conn.cursor().execute(
524                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
525                 conn.commit()
526             return True
527         except Exception as e:
528             logger.error(f"Failed to set progress for job {job_id}: {e}")
529             return False
530
531     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
532         try:

```

```

533         with self._connect() as conn:
534             conn.cursor().execute(
535                 "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
536             conn.commit()
537         return True
538     except Exception as e:
539         logger.error(f"Failed to set video_id for job {job_id}: {e}")
540         return False
541
542     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
543         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
544         validation) is inside `result` ? job status 'done' means 'the worker
545         finished'."""
546         try:
547             with self._connect() as conn:
548                 conn.cursor().execute(
549                     "UPDATE jobs SET status='done', progress='finished', result=?, "
550                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
551                     (json.dumps(result), job_id))
552                 conn.commit()
553             return True
554         except Exception as e:
555             logger.error(f"Failed to mark job {job_id} done: {e}")
556             return False
557
558     def mark_job_failed(self, job_id: str, error: str) -> bool:
559         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
560         try:
561             with self._connect() as conn:
562                 conn.cursor().execute(
563                     "UPDATE jobs SET status='failed', error=?, "
564                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
565                 conn.commit()
566             return True
567         except Exception as e:
568             logger.error(f"Failed to mark job {job_id} failed: {e}")
569             return False
570
571     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
572         """Fetch one job; `result`/`player_pool` JSON deserialized."""
573         with self._connect() as conn:
574             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
575             (job_id,)).fetchone()
576             if not row:
577                 return None
578             d = dict(row)
579             d["result"] = json.loads(d["result"]) if d["result"] else None
580             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
581             None
582             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
583             return d
584
585     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
586     -----
587
588     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
589         """Attach/replace a job's JSON ExecutionPolicy."""
590         try:
591             with self._connect() as conn:
592                 conn.cursor().execute(
593                     "UPDATE jobs SET policy=? WHERE id = ?",
594                     (json.dumps(policy) if policy is not None else None, job_id))
595                 conn.commit()
596             return True
597         except Exception as e:
598             logger.error(f"Failed to set policy for job {job_id}: {e}")

```

```

594         return False
595
596     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
597         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
598         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
599         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
600         try:
601             with self._connect() as conn:
602                 conn.cursor().execute(
603                     "UPDATE jobs SET status='waiting', "
604                     "next_poll_at=datetime('now', ?), "
605                     "progress=COALESCE(?, progress) WHERE id = ?",
606                     (f"{max(0, int(delay_sec))} seconds", progress, job_id))
607                 conn.commit()
608             return True
609         except Exception as e:
610             logger.error(f"Failed to set job {job_id} waiting: {e}")
611             return False
612
613     def seconds_until_next_due(self) -> Optional[float]:
614         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
615         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
616         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
617         try:
618             with self._connect() as conn:
619                 row = conn.cursor().execute(
620                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
621                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
622                     "FROM jobs WHERE status='waiting'").fetchone()
623                 if row is None or row["secs"] is None:
624                     return None
625                 return max(0.0, float(row["secs"]))
626         except Exception as e:
627             logger.error(f"Failed to compute next-due delay: {e}")
628             return None
629
630     def claim_due_job(self) -> Optional[Dict[str, Any]]:
631         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
632         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
633         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
634         first. Returns the claimed job dict, or None if nothing is runnable."""
635         try:
636             with self._connect() as conn:
637                 cur = conn.cursor()
638                 row = cur.execute(
639                     "SELECT id, status FROM jobs WHERE status='queued' "
640                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
641                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
642                 if not row:
643                     return None
644                 cur.execute(
645                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "

```

```

646         "WHERE id=? AND status=?", (row["id"], row["status"]))
647         conn.commit()
648         if cur.rowcount != 1:
649             return None                    # lost the race to a concurrent
worker
650         return self.get_job(row["id"])
651     except Exception as e:
652         logger.error(f"Failed to claim a due job: {e}")
653         return None
654
655     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
656         """Crash recovery: any job still queued/running from a previous process is dead
657         (the executor is in-process). Mark failed so clients see a terminal state, not a
658         hang. Called once at startup. Returns the number of jobs swept."""
659         try:
660             with self._connect() as conn:
661                 cur = conn.cursor()
662                 cur.execute(
663                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
664                     "WHERE status IN ('queued', 'running')", (reason,))
665                 conn.commit()
666                 return cur.rowcount
667         except Exception as e:
668             logger.error(f"Failed to sweep stale jobs: {e}")
669             return 0
670
671     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
672     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
673     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
674     # serving.py) resolves them, and wiring that bridge into the production decision
seam
675     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
676     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
677     # JSON-serialised dicts; the rest are scalar columns.
678
679     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
680                           baseline_version: Optional[str] = None,
681                           feature_schema_hash: Optional[str] = None,
682                           fusion_config: Optional[Dict[str, Any]] = None,
683                           classifier_json: Optional[Dict[str, Any]] = None,
684                           promotion_evidence: Optional[Dict[str, Any]] = None,
685                           n_videos_at_fit: int = 0,
686                           is_active: bool = False) -> Optional[int]:
687         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
688
689         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
690         rowid). When `is_active` is True this row becomes the user's single active
model ?
691         any previously-active row for the SAME user is first cleared, so the
692         `idx_user_models_one_active` partial-unique invariant always holds (one active
model
693         per user). Returns the row id, or None on failure.
694         """
695         try:
696             with self._connect() as conn:
697                 cur = conn.cursor()
698                 if is_active:
699                     # Demote the current active row first (NULL user_id = local: IS NULL
match).
700                     if user_id is None:
701                         cur.execute(
702                             "UPDATE user_models SET is_active = 0 "
703                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",

```

```

704         (version,))
705     else:
706         cur.execute(
707             "UPDATE user_models SET is_active = 0 "
708             "WHERE user_id = ? AND is_active = 1 AND version != ?",
709             (user_id, version))
710     cur.execute(
711         """INSERT INTO user_models
712         (user_id, version, baseline_version, feature_schema_hash,
713         fusion_config, classifier_json, promotion_evidence,
714         n_videos_at_fit, is_active)
715         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
716         ON CONFLICT(user_id, version) DO UPDATE SET
717         baseline_version=excluded.baseline_version,
718         feature_schema_hash=excluded.feature_schema_hash,
719         fusion_config=excluded.fusion_config,
720         classifier_json=excluded.classifier_json,
721         promotion_evidence=excluded.promotion_evidence,
722         n_videos_at_fit=excluded.n_videos_at_fit,
723         is_active=excluded.is_active""",
724         (user_id, version, baseline_version, feature_schema_hash,
725         json.dumps(fusion_config) if fusion_config is not None else None,
726         json.dumps(classifier_json) if classifier_json is not None else
None,
727         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
728         n_videos_at_fit, 1 if is_active else 0))
729     conn.commit()
730     rid = cur.execute(
731         "SELECT id FROM user_models WHERE "
732         + ("user_id IS NULL" if user_id is None else "user_id = ?")
733         + " AND version = ?",
734         ((version,) if user_id is None else (user_id, version))).fetchone()
735     return int(rid[0]) if rid else None
736 except Exception as e:
737     logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
738     return None
739
740 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
741     """Return the single active per-user model row for ``user_id`` (NULL = local
install),
742     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
743     or None if the user has no active model. Does not touch the served pipeline."""
744     with self._connect() as conn:
745         if user_id is None:
746             row = conn.cursor().execute(
747                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
748         else:
749             row = conn.cursor().execute(
750                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
751                 (user_id,)).fetchone()
752         if not row:
753             return None
754         d = dict(row)
755         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
756             d[col] = json.loads(d[col]) if d[col] else None
757         return d
758

```

6. assistant (2026-06-18T10:21:16.311Z)

Now let me identify the exact methods to route. Let me catalog the bool-returning single-

statement writers:

****Routed (bool-returning, single-statement writes):****

1. ``add_video` ? "Failed to add video: {e}"`
2. ``add_event` ? "Failed to add event: {e}"`
3. ``link_candidate_to_segment` ? "Failed to link candidate {candidate_id} to segment {segment_id}: {e}"`
4. ``create_job` ? "Failed to create job {job_id}: {e}"`
5. ``mark_job_running` ? "Failed to mark job {job_id} running: {e}"`
6. ``set_job_progress` ? "Failed to set progress for job {job_id}: {e}"`
7. ``set_job_video_id` ? "Failed to set video_id for job {job_id}: {e}"`
8. ``mark_job_done` ? "Failed to mark job {job_id} done: {e}"`
9. ``mark_job_failed` ? "Failed to mark job {job_id} failed: {e}"`
10. ``set_job_policy` ? "Failed to set policy for job {job_id}: {e}"`
11. ``set_job_waiting` ? "Failed to set job {job_id} waiting: {e}"`

****Excluded (lastrowid):**** `add_play_segment, add_candidate_segment, upsert_user_model, claim_due_job.`

****Excluded (multi-statement/non-bool):**** `prune_segments (None), fail_stale_jobs (int), seconds_until_next_due (reads).`

Note ``seconds_until_next_due`` returns `Optional[float]` (a read). Let me check existing tests for this module to mirror style.

7. user (2026-06-18T10:21:16.846Z)

`tests\test_database.py`

8. user (2026-06-18T10:21:17.077Z)

`scratch\test_ai_segmenter.py`
`scratch\test_thresholds.py`
`tests\test_ablation.py`
`tests\test_annotation_providers.py`
`tests\test_api_endpoints.py`
`tests\test_async_jobs.py`
`tests\test_audio_features.py`
`tests\test_automated_provider.py`
`tests\test_base.py`
`tests\test_calibrate_local.py`
`tests\test_calibrate_wasb.py`
`tests\test_calibration.py`
`tests\test_cleanup_caches.py`
`tests\test_compiler.py`
`tests\test_config.py`
`tests\test_config_wasb.py`
`tests\test_database.py`
`tests\test_detector_keying.py`
`tests\test_distill_local.py`
`tests\test_eval_annotations.py`
`tests\test_eval_batch.py`
`tests\test_eval_classifier.py`
`tests\test_eval_harness.py`
`tests\test_eval_metrics.py`
`tests\test_eval_partial_labels.py`
`tests\test_eval_regression.py`
`tests\test_eval_stats.py`
`tests\test_eval_training.py`
`tests\test_execution_policy.py`
`tests\test_experiment.py`
`tests\test_experiment_honest.py`
`tests\test_export_wasb_segments.py`
`tests\test_fusion_compare.py`
`tests\test_fusion_features.py`
`tests\test_fusion_golden_telemetry.py`

tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_gemini_refine.py
tests\test_generate_highlights.py
tests\test_geometry.py
tests\test_golden_fixtures.py
tests\test_golden_real_fixtures.py
tests\test_gt_loader.py
tests\test_hashing.py
tests\test_hit_detector.py
tests\test_import_smoke.py
tests\test_job_self_scheduling.py
tests\test_job_worker_loop.py
tests\test_logging_setup.py
tests\test_nightly_regression.py
tests\test_path_safety.py
tests\test_per_user_eval.py
tests\test_per_user_gate.py
tests\test_per_user_regression.py
tests\test_personalization_serving.py
tests\test_person_detector.py
tests\test_promotion.py
tests\test_provider_gemini.py
tests\test_providers.py
tests\test_rally_gate.py
tests\test_rally_seg_eval.py
tests\test_rally_seq_proto.py
tests\test_rally_slicer.py
tests\test_rally_summary.py
tests\test_reingest_safety.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_reporting_harvest.py
tests\test_reporting_spec_faq.py
tests\test_results.py
tests\test_run_signals.py
tests\test_run_telemetry.py
tests\test_score_calibration.py
tests\test_segmentation_metrics.py
tests\test_serve_contrast.py
tests\test_served_gate_a_regression.py
tests\test_splits.py
tests\test_sport_profiles.py
tests\test_tolerance_metrics.py
tests\test_tracknet_hybrid.py
tests\test_tracknet_runner.py
tests\test_training_gen0.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_upload_download.py
tests\test_validators_base.py
tests\test_variant_tuning.py
tests\test_video.py
tests\test_wasb_cache.py
tests\test_wasb_runner.py
tests\test_watermark.py
tests\test_windowing.py
tests\test_wsl_tilde_quoting.py
tests\test_yolo_hybrid.py

9. user (2026-06-18T10:21:20.107Z)

```
1     import sqlite3
2     import pytest
3     from backend.infrastructure.database import Database
4
```

```

5     def test_database_init(tmp_path):
6         db_path = tmp_path / "test.db"
7         Database(str(db_path)) # instantiation creates the schema
8
9         conn = sqlite3.connect(str(db_path))
10        cursor = conn.cursor()
11        cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
12        tables = [r[0] for r in cursor.fetchall()]
13        assert "videos" in tables
14        assert "play_segments" in tables
15        conn.close()
16
17    def test_database_video(tmp_path):
18        db = Database(str(tmp_path / "test.db"))
19
20        vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
21        assert vid_added is True
22
23        video = db.get_video("vid1")
24        assert video["id"] == "vid1"
25        assert video["filepath"] == "vid1.mp4"
26        assert video["status"] == "pending"
27        assert video["validation_results"] == [{"val": 1}]
28
29        db.clear_video_data("vid1")
30        assert db.get_video("vid1") is None
31
32    def test_database_play_segment(tmp_path):
33        db = Database(str(tmp_path / "test.db"))
34        db.add_video("vid1", "vid1.mp4", "pending", [])
35
36        sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
37        assert sid is not None
38
39        db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
40
41        segments = db.query_play_segments("vid1", {"server": "PlayerA"})
42        assert len(segments) == 1
43        assert segments[0]["id"] == sid
44        assert segments[0]["start_time"] == 10.0
45        assert segments[0]["server"] == "PlayerA"
46        assert segments[0]["metadata"]["foo"] == "bar"
47
48        assert len(segments[0]["events"]) == 1
49        assert segments[0]["events"][0]["type"] == "smash"
50        assert segments[0]["events"][0]["details"]["speed"] == 300
51
52
53    # --- candidate_segments schema + behavior -----
54
55    # The schema contract: any future change to candidate_segments columns must
56    # consciously update this set, otherwise the test below fails (regression guard).
57    EXPECTED_CANDIDATE_COLUMNS = {
58        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
59        "duration", "confidence", "stats", "detection_params",
60        "confirmed_segment_id", "human_label", "notes", "created_at",
61        "user_id", # v4: personalization groundwork (nullable; NULL = local install)
62    }
63
64
65    def test_candidate_schema_contract(tmp_path):
66        Database(str(tmp_path / "test.db")) # instantiation creates the schema
67        conn = sqlite3.connect(str(tmp_path / "test.db"))
68        cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69        conn.close()

```

```

70
71     names = {c[1] for c in cols}
72     assert names == EXPECTED_CANDIDATE_COLUMNS
73
74     # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
75     by_name = {c[1]: c for c in cols}
76     assert by_name["id"][5] == 1 # primary key
77     for col in ("video_id", "detector", "start_time", "end_time", "duration"):
78         assert by_name[col][3] == 1, f"{col} should be NOT NULL"
79
80
81     def test_schema_version_is_current(tmp_path):
82         db_path = tmp_path / "test.db"
83         Database(str(db_path))
84         conn = sqlite3.connect(str(db_path))
85         version = conn.execute("PRAGMA user_version").fetchone()[0]
86         conn.close()
87         # v5 = per-user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
88         assert version == 5
89
90
91     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
92         db_path = tmp_path / "legacy.db"
93         # Simulate a pre-v1 database that only has the old tables and no version.
94         conn = sqlite3.connect(str(db_path))
95         conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
96         conn.commit()
97         conn.close()
98
99         # Opening with the new Database should add candidate_segments without error...
100         Database(str(db_path))
101         # ...and running init again must be a no-op (idempotent).
102         Database(str(db_path))
103
104         conn = sqlite3.connect(str(db_path))
105         tables = [r[0] for r in conn.execute(
106             "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
107         version = conn.execute("PRAGMA user_version").fetchone()[0]
108         conn.close()
109         assert "candidate_segments" in tables
110         assert "jobs" in tables # v2 (async job model, #16)
111         assert "user_models" in tables # v5 (per-user model store)
112         assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
113
114
115     def test_candidate_round_trip(tmp_path):
116         db = Database(str(tmp_path / "test.db"))
117         db.add_video("vid1", "vid1.mp4", "processed", [])
118
119         cid = db.add_candidate_segment(
120             "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
121             stats={"peak_velocity": 0.8, "track_id_count": 2},
122             detection_params={"frame_skip": 3},
123         )
124         assert cid is not None
125
126         cands = db.query_candidate_segments("vid1")
127         assert len(cands) == 1
128         c = cands[0]
129         assert c["detector"] == "yolo_hybrid"
130         assert c["start_time"] == 10.0
131         assert c["duration"] == 15.0
132         assert c["confidence"] == 0.42

```

```

133     assert c["stats"]["peak_velocity"] == 0.8
134     assert c["detection_params"]["frame_skip"] == 3
135     assert c["confirmed_segment_id"] is None
136
137     # detector filter
138     assert db.query_candidate_segments("vid1", detector="motion") == []
139     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
140
141
142     def test_candidate_link_and_unlabeled_filter(tmp_path):
143         db = Database(str(tmp_path / "test.db"))
144         db.add_video("vid1", "vid1.mp4", "processed", [])
145
146         cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
147         sid = db.add_play_segment("vid1", 1.0, 4.0)
148
149         assert db.link_candidate_to_segment(cid, sid) is True
150         assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
151
152         # only_unlabeled returns it while human_label is NULL
153         assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
154         with db._get_connection() as conn:
155             conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
156                          ("true_rally", cid))
157             conn.commit()
158         assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
159
160
161     def test_candidate_cascade_on_video_delete(tmp_path):
162         db = Database(str(tmp_path / "test.db"))
163         db.add_video("vid1", "vid1.mp4", "processed", [])
164         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
165
166         db.clear_video_data("vid1")
167         assert db.query_candidate_segments("vid1") == []
168
169
170     # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) -----
171
172     def _table_cols(db_path, table):
173         conn = sqlite3.connect(str(db_path))
174         cols = [c[1] for c in conn.execute(f"PRAGMA table_info({table})").fetchall()]
175         conn.close()
176         return cols
177
178
179     def test_v4_user_id_columns_present_and_nullable(tmp_path):
180         db_path = tmp_path / "test.db"
181         Database(str(db_path))
182         assert "user_id" in _table_cols(db_path, "videos")
183         assert "user_id" in _table_cols(db_path, "candidate_segments")
184
185         # The column is NULLABLE: PRAGMA table_info notnull flag must be 0.
186         conn = sqlite3.connect(str(db_path))
187         for table in ("videos", "candidate_segments"):
188             by_name = {c[1]: c for c in conn.execute(f"PRAGMA
table_info({table})").fetchall()}
189             assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
190             conn.close()
191
192
193     def test_v4_existing_writers_leave_user_id_null(tmp_path):
194         """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
195         row a current writer produces is local (NULL) ? byte-identical to pre-v4

```

```

behaviour."""
196     db_path = tmp_path / "test.db"
197     db = Database(str(db_path))
198     db.add_video("vid1", "vid1.mp4", "processed", [])
199     db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
200
201     conn = sqlite3.connect(str(db_path))
202     vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
203     cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
204     conn.close()
205     assert vu is None
206     assert cu is None
207     # Existing readers are unaffected (no user_id filter in the query path).
208     assert len(db.query_candidate_segments("vid1")) == 1
209
210
211 def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
212     """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the
new
213     columns/tables appear, user_id back-fills to NULL, and prior rows survive the
ALTERs."""
214     db_path = tmp_path / "legacy_v3.db"
215     db = Database(str(db_path)) # fresh DB is born at the current version; simulate a
v3 state
216     # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
217     db.add_video("vidA", "a.mp4", "processed", [{"v": 1}])
218     db.add_candidate_segment("vidA", "wasb_hybrid", 2.0, 9.0, stats={"net_crossings":
3})
219     conn = sqlite3.connect(str(db_path))
220     conn.execute("DROP INDEX IF EXISTS idx_videos_user")
221     conn.execute("DROP INDEX IF EXISTS idx_candidates_user")
222     conn.execute("DROP TABLE IF EXISTS user_models")
223     # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind
version
224     # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
225     conn.execute("ALTER TABLE videos DROP COLUMN user_id")
226     conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
227     conn.execute("PRAGMA user_version = 3")
228     conn.commit()
229     conn.close()
230     assert "user_id" not in _table_cols(db_path, "videos")
231
232     # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
233     db2 = Database(str(db_path))
234     conn = sqlite3.connect(str(db_path))
235     assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
236     assert "user_models" in {r[0] for r in conn.execute(
237         "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
238     conn.close()
239     assert "user_id" in _table_cols(db_path, "videos")
240     assert "user_id" in _table_cols(db_path, "candidate_segments")
241
242     v = db2.get_video("vidA")
243     assert v is not None and v["validation_results"] == [{"v": 1}]
244     cands = db2.query_candidate_segments("vidA")
245     assert len(cands) == 1 and cands[0]["stats"]["net_crossings"] == 3
246
247
248 def test_migration_idempotent_on_interrupted_upgrade(tmp_path):
249     """An INTERRUPTED v3/v4 upgrade must not brick the DB. A DDL ``ALTER`` auto-commits
250     on its own, so a crash after the ADD COLUMNS but before the ``PRAGMA user_version``
251     bump leaves the columns present while the version is stale ? re-opening re-runs the
252     bare ALTERs. The ladder must tolerate that (re-runnable, like the v1/v2/v5 blocks)
253     instead of raising ``OperationalError: duplicate column name``."""

```

```

254     db_path = tmp_path / "interrupted.db"
255     Database(str(db_path)) # fully migrate to the current version (v3/v4 columns all
present)
256
257     # Simulate the interruption: the v3/v4 ADD COLUMNs already landed and auto-
committed,
258     # but the process died before the version bump, so user_version is stuck at 2.
259     conn = sqlite3.connect(str(db_path))
260     conn.execute("DROP TABLE IF EXISTS user_models")          # a true v2 DB lacks the
v5 table
261     conn.execute("PRAGMA user_version = 2")
262     conn.commit()
263     conn.close()
264
265     # Re-opening re-runs v3 (jobs.policy/next_poll_at) and v4 (videos/candidate_segments
266     # .user_id) over columns that ALREADY exist ? must NOT raise, and must reach v5.
267     Database(str(db_path))
268     conn = sqlite3.connect(str(db_path))
269     assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
270     conn.close()
271     # Columns/tables are all present after the clean re-apply.
272     assert "policy" in _table_cols(db_path, "jobs")
273     assert "user_id" in _table_cols(db_path, "videos")
274     assert "user_id" in _table_cols(db_path, "candidate_segments")
275     conn = sqlite3.connect(str(db_path))
276     assert "user_models" in {r[0] for r in conn.execute(
277         "SELECT name FROM sqlite_master WHERE type='table'").fetchall()}
278     conn.close()
279
280
281     def test_add_column_idempotent_swallows_only_duplicate(tmp_path):
282         """The idempotent ADD COLUMN helper swallows ONLY 'duplicate column name'. Any other
283         OperationalError (e.g. a genuinely broken migration against a missing table) must
still
284         propagate ? the swallow is narrow, not a blanket except."""
285         db_path = tmp_path / "helper.db"
286         Database(str(db_path))          # current schema; jobs.policy already
exists
287         conn = sqlite3.connect(str(db_path))
288         cur = conn.cursor()
289         # Re-adding an existing column is tolerated (the crash-safety path).
290         Database._add_column_idempotent(cur, "ALTER TABLE jobs ADD COLUMN policy TEXT")
291         # An unrelated OperationalError (no such table) is NOT masked.
292         with pytest.raises(sqlite3.OperationalError):
293             Database._add_column_idempotent(cur, "ALTER TABLE nope ADD COLUMN x TEXT")
294         conn.close()
295
296
297     # --- v5: per-user model store CRUD -----
298
299     def test_user_models_schema_contract(tmp_path):
300         db_path = tmp_path / "test.db"
301         Database(str(db_path))
302         cols = _table_cols(db_path, "user_models")
303         assert cols == {
304             "id", "user_id", "version", "baseline_version", "feature_schema_hash",
305             "fusion_config", "classifier_json", "promotion_evidence",
306             "n_videos_at_fit", "is_active", "created_at",
307         }
308
309
310     def test_upsert_and_get_active_user_model_round_trip(tmp_path):
311         db = Database(str(tmp_path / "test.db"))
312         rid = db.upsert_user_model(
313             "user-1", version=1, baseline_version="base-2026-06",

```

```

314         feature_schema_hash="abc123",
315         fusion_config={"min_crossings": 3, "cue_flags": {"person": True}},
316         classifier_json={"model": "logreg", "w": [0.1, 0.2], "b": 0.0},
317         promotion_evidence={"promote": True, "n": 6},
318         n_videos_at_fit=6, is_active=True,
319     )
320     assert rid is not None
321
322     m = db.get_active_user_model("user-1")
323     assert m is not None
324     assert m["user_id"] == "user-1"
325     assert m["version"] == 1
326     assert m["baseline_version"] == "base-2026-06"
327     assert m["feature_schema_hash"] == "abc123"
328     assert m["fusion_config"] == {"min_crossings": 3, "cue_flags": {"person": True}}
329     assert m["classifier_json"]["w"] == [0.1, 0.2]
330     assert m["promotion_evidence"]["promote"] is True
331     assert m["n_videos_at_fit"] == 6
332     assert m["is_active"] == 1
333
334
335     def test_get_active_user_model_none_when_absent_or_inactive(tmp_path):
336         db = Database(str(tmp_path / "test.db"))
337         assert db.get_active_user_model("nobody") is None
338
339         # An inactive model is not returned as active.
340         db.upsert_user_model("user-2", version=1, is_active=False)
341         assert db.get_active_user_model("user-2") is None
342
343
344     def test_upsert_is_idempotent_on_user_version(tmp_path):
345         db = Database(str(tmp_path / "test.db"))
346         rid1 = db.upsert_user_model("user-3", version=1, n_videos_at_fit=4, is_active=True)
347         rid2 = db.upsert_user_model(
348             "user-3", version=1, n_videos_at_fit=7, is_active=True,
349             fusion_config={"min_crossings": 5},
350         )
351         # Same (user, version) ? same row updated in place, not a duplicate.
352         assert rid1 == rid2
353         m = db.get_active_user_model("user-3")
354         assert m["n_videos_at_fit"] == 7
355         assert m["fusion_config"] == {"min_crossings": 5}
356
357         conn = sqlite3.connect(str(tmp_path / "test.db"))
358         count = conn.execute(
359             "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
360         conn.close()
361         assert count == 1
362
363
364     def test_activating_new_version_demotes_prior_active(tmp_path):
365         db = Database(str(tmp_path / "test.db"))
366         db.upsert_user_model("user-4", version=1, n_videos_at_fit=5, is_active=True)
367         db.upsert_user_model("user-4", version=2, n_videos_at_fit=9, is_active=True)
368
369         active = db.get_active_user_model("user-4")
370         assert active["version"] == 2 # the newer model is the single active one
371
372         # Exactly one active row for the user (the partial-unique invariant holds).
373         conn = sqlite3.connect(str(tmp_path / "test.db"))
374         n_active = conn.execute(
375             "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
376             1").fetchone()[0]
377         conn.close()
378         assert n_active == 1

```

```

378
379
380     def test_null_user_id_local_install_round_trip(tmp_path):
381         """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
382         '=NULL')."""
383         db = Database(str(tmp_path / "test.db"))
384         db.upsert_user_model(None, version=1, fusion_config={"min_crossings": 2},
385                             n_videos_at_fit=5, is_active=True)
386         m = db.get_active_user_model(None)
387         assert m is not None
388         assert m["user_id"] is None
389         assert m["fusion_config"] == {"min_crossings": 2}
390         # A named user does NOT see the local row.
391         assert db.get_active_user_model("user-x") is None
392
393     def test_user_model_json_columns_default_none(tmp_path):
394         """A config-only model (no classifier) stores NULL classifier_json and deserialises
395         to None."""
396         db = Database(str(tmp_path / "test.db"))
397         db.upsert_user_model("user-5", version=1, fusion_config={"min_crossings": 4},
398                             n_videos_at_fit=5, is_active=True)
399         m = db.get_active_user_model("user-5")
400         assert m["classifier_json"] is None
401         assert m["promotion_evidence"] is None
402         assert m["fusion_config"] == {"min_crossings": 4}
403

```

10. assistant (2026-06-18T10:21:23.082Z)

Now I understand the existing test style. Let me check existing failure-branch coverage in the async jobs test to understand how errors are forced.

11. user (2026-06-18T10:21:25.486Z)

```

35: def env(tmp_path, monkeypatch):
36:     monkeypatch.setattr(m, "db_instance", db)
37:     monkeypatch.setattr(m, "global_config", cfg)
38:     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
39:     return client, db, tmp_path, monkeypatch
40:
41: with db._get_connection() as conn:
42:     with db._get_connection() as c:
43:         def test_parity_job_model_with_direct_processing(env, tmp_path, monkeypatch):
44:             with db._get_connection() as c:

```

12. assistant (2026-06-18T10:21:27.813Z)

Let me check how existing tests force write failures to find the cleanest mock pattern.

13. user (2026-06-18T10:21:29.574Z)

```

32:     conn = db._get_connection()

```

14. assistant (2026-06-18T10:21:33.646Z)

Let me check whether any existing test exercises the return-False branch of these writers, and how caplog is used in the repo.

15. user (2026-06-18T10:21:34.937Z)

Found 12 files
tests\test_watermark.py
tests\test_tracknet_runner.py
tests\test_provider_gemini.py


```
tests\test_gt_loader.py
tests\test_gemini.py
tests\test_fusion_golden_telemetry.py
tests\test_experiment.py
tests\test_execution_policy.py
tests\test_database.py
tests\test_config.py
tests\test_automated_provider.py
tests\test_base.py
```

16. assistant (2026-06-18T10:21:36.857Z)

Wait, test_database.py already uses caplog? Let me check.

17. user (2026-06-18T10:21:38.116Z)

No matches found

18. assistant (2026-06-18T10:21:40.675Z)

The grep matched substrings in other terms. Let me look at how caplog + the database logger is used in test_execution_policy.py as a model for asserting log messages.

19. user (2026-06-18T10:21:42.171Z)

```
5:are harmless). Logging discipline (polls at DEBUG, transitions at INFO) is asserted via
caplog.
7:import logging
21:LOGGER = logging.getLogger("test.execution.policy")
61:     assert run_bounded(lambda: 42, DEFAULT_POLICY, label="ok", sleep=clock.sleep,
logger=LOGGER) == 42
76:     assert run_bounded(flaky, pol, label="flaky", sleep=clock.sleep, logger=LOGGER) == "ok"
89:         run_bounded(always_fail, pol, label="bad", sleep=clock.sleep, logger=LOGGER)
97:         run_bounded(lambda: time.sleep(2.0), pol, label="hang", sleep=clock.sleep,
logger=LOGGER)
111:     assert run_bounded(hang_once, pol, label="h", sleep=clock.sleep, logger=LOGGER) ==
"recovered"
125:         run_bounded(fail, pol, label="abort", sleep=clock.sleep, logger=LOGGER)
134:             sleep=clock.sleep, monotonic=clock.monotonic, logger=LOGGER) == "done"
148:             logger=LOGGER) == "value"
157:             monotonic=clock.monotonic, logger=LOGGER)
161:def test_poll_logging_is_debug_not_info(caplog):
170:     with caplog.at_level(logging.DEBUG, logger="test.execution.policy"):
172:         monotonic=clock.monotonic, logger=LOGGER)
173:     debug = [r for r in caplog.records if r.levelno == logging.DEBUG and "poll" in
r.message]
174:     info = [r for r in caplog.records if r.levelno == logging.INFO]
```

20. assistant (2026-06-18T10:21:49.541Z)

Now I understand the caplog pattern. The database module's logger is
`backend.infrastructure.database`.

Now let me write characterization tests. I'll force a write to raise `sqlite3.Error` by monkeypatching `\_connect` to yield a connection whose cursor's `execute` raises ? but the cleanest approach that pins the *exact* per-method log message is to pass a value that genuinely fails at the SQL layer.

The most robust deterministic approach: monkeypatch `\_get\_connection` so that the cursor raises `sqlite3.Error` on execute. Let me design it to capture the message exactly via caplog. I'll write tests for at least 2 routed methods (`mark\_job\_running` and `create\_job`, plus `add\_video`, `add\_event` for breadth). Let me verify the logger name and that `propagate` is default True.

21. user (2026-06-18T10:21:51.073Z)

```
backend.infrastructure.database
propagate True
```

22. assistant (2026-06-18T10:21:55.351Z)

Now I'll write the characterization tests. I'll force a `sqlite3.Error` by monkeypatching `db.\_get\_connection` to return a connection whose cursor raises on `execute`. This deterministically drives the except branch for the routed bool-returning writers, and I'll assert both the return value (`False`) and the exact per-method log message via caplog.

I'll cover 4 routed methods to exceed the "at least 2" requirement, including ones with format args (`create\_job` with `{job\_id}`, `link\_candidate\_to\_segment` with two args), since those are the ones most likely to silently change if the error_msg routing is mishandled.

23. user (2026-06-18T10:21:56.371Z)

```
1     import sqlite3
2     import pytest
3     from backend.infrastructure.database import Database
4
```

24. user (2026-06-18T10:21:59.585Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

25. assistant (2026-06-18T10:22:01.736Z)

Now append the characterization tests at the end of the file.

26. user (2026-06-18T10:22:18.278Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-18T10:22:20.607Z)

Now run these characterization tests against the UNCHANGED production code to confirm they pass.

28. user (2026-06-18T10:22:24.048Z)

```
..... [100%]
6 passed, 21 deselected in 0.28s
```

29. assistant (2026-06-18T10:22:26.604Z)

All 6 characterization tests pass against unchanged code. Let me also run the full test_database.py file to confirm nothing else broke.

30. user (2026-06-18T10:22:29.495Z)

```
..... [100%]
27 passed in 1.01s
```

31. assistant (2026-06-18T10:22:38.964Z)

All green pre-refactor (21 existing + 6 new). Now the behavior-preserving refactor. I'll introduce `\_execute\_write` and route the 11 bool-returning single-statement writers through it, preserving each method's exact log message text by passing it as `error\_msg`.

Let me add the helper after `\_get\_connection` (and before `\_add\_column\_idempotent` to keep connection-related helpers grouped). I'll place it right after `\_get\_connection`.

32. user (2026-06-18T10:22:44.641Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-06-18T10:22:48.150Z)

A subtlety: the original methods log `f"Failed to add video: {e}"` ? i.e. `

34. user (2026-06-18T10:22:51.873Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. user (2026-06-18T10:22:57.808Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

36. user (2026-06-18T10:23:02.534Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

37. user (2026-06-18T10:23:08.616Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

38. user (2026-06-18T10:23:13.288Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. user (2026-06-18T10:23:17.670Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

40. user (2026-06-18T10:23:22.315Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. user (2026-06-18T10:23:27.368Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

42. user (2026-06-18T10:23:35.332Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

43. user (2026-06-18T10:23:40.626Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. user (2026-06-18T10:23:47.092Z)

The file C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/wf_1d39e81c-851-3/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

45. assistant (2026-06-18T10:23:49.936Z)

All 11 routed. Let me verify the excluded methods are untouched and confirm there are no leftover `return True` bool-writers I missed by re-reading the relevant region.

46. user (2026-06-18T10:23:51.329Z)

```
44:     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
47:         Captures the connect/commit/except-logger.error/return-False boilerplate shared by
54:             conn.cursor().execute(sql, params)
56:             return True
58:             logger.error(f"{error_msg}: {e}")
297:         return self._execute_write(
352:             logger.error(f"Failed to add play segment: {e}")
356:         return self._execute_write(
382:             logger.error(f"Failed to add candidate segment: {e}")
387:         return self._execute_write(
410:             rows = conn.cursor().execute(query, params).fetchall()
420:             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
479:             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
483:         # Writers follow the repo convention (swallow + logger.error, return bool); the
492:         return self._execute_write(
502:         return self._execute_write(
509:         return self._execute_write(
515:         return self._execute_write(
523:         return self._execute_write(
532:         return self._execute_write(
541:             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
553:         return self._execute_write(
563:         return self._execute_write(
577:             row = conn.cursor().execute(
585:             logger.error(f"Failed to compute next-due delay: {e}")
610:             logger.error(f"Failed to claim a due job: {e}")
626:             logger.error(f"Failed to sweep stale jobs: {e}")
695:             logger.error(f"Failed to upsert user_model (user={user_id}, v={version}): {e}")
704:             row = conn.cursor().execute(
707:             row = conn.cursor().execute(
```

47. assistant (2026-06-18T10:23:55.421Z)

All 11 routed methods now use `\_execute\_write`. The remaining `logger.error` lines are the excluded methods (add_play_segment, add_candidate_segment lastrowid; fail_stale_jobs int; seconds_until_next_due/claim_due_job reads; upsert_user_model lastrowid) ? all correctly untouched. `prune\_segments`, `clear\_video\_data`, `segment\_watermarks` (no error handling /

multi-statement / reads) untouched.

Now re-verify the same characterization tests pass unchanged, plus the full database test file.

48. user (2026-06-18T10:23:58.458Z)

```
..... [100%]  
27 passed in 0.99s
```

49. assistant (2026-06-18T10:24:01.702Z)

All 27 pass post-refactor (unchanged). Let me run the related async/job/execution-policy tests that exercise these writers in success paths, then the full gate.

50. user (2026-06-18T10:24:14.642Z)

```
..... [ 97%]  
..  
74 passed in 9.56s [100%]
```

51. assistant (2026-06-18T10:24:17.284Z)

All green. Now the full binding gate. Step 1: full suite.

52. user (2026-06-18T10:25:27.909Z)

```
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest  
-q  
obsreport (optional reporting dep): available
```

```
=====
```

.....SS.....	[8%]
.....	[16%]
.....	[24%]
.....	[33%]
.....	[41%]
.....	[49%]
.....	[57%]
.....	[66%]
.....	[74%]
.....SSSSS.....	[82%]
.....	[91%]
.....	[99%]
.....	[100%]

```
863 passed, 7 skipped in 66.89s (0:01:06)
```

```
=====
```

```
RESULT: PASS - all tests green
```

```
=====
```

53. assistant (2026-06-18T10:25:30.666Z)

863 passed, 7 skipped. Now ruff, mypy, and coverage.

54. user (2026-06-18T10:26:12.743Z)

```
=== RUFF ===  
All checks passed!  
=== MYPY ===  
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not  
checked, consider using --check-untyped-defs [annotation-unchecked]  
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,  
consider using --check-untyped-defs [annotation-unchecked]  
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not  
checked, consider using --check-untyped-defs [annotation-unchecked]  
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
```

checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 107 source files

55. assistant (2026-06-18T10:26:15.273Z)

Ruff clean, mypy clean (those are pre-existing informational notes, "Success: no issues found").
Now coverage gate.

56. user (2026-06-18T10:27:58.571Z)

backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%
backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8024	1509	81%

Required test coverage of 70% reached. Total coverage: 81.19%
863 passed, 7 skipped, 7 warnings in 98.46s (0:01:38)
=====

RESULT: PASS - all tests green

=====

57. assistant (2026-06-18T10:28:02.296Z)

All four gates green. Coverage is 81.19%, which is above the ~80.9% base ? not a regression (the dedup plus 6 new failure-branch tests improved database.py coverage). Let me confirm the database.py module coverage specifically and check the diff.

58. user (2026-06-18T10:28:04.587Z)

```
backend/infrastructure/database.py | 200 ++++++-----
tests/test_database.py             | 98 ++++++
2 files changed, 177 insertions(+), 121 deletions(-)
===NUMSTAT===
79      121      backend/infrastructure/database.py
98       0       tests/test_database.py
```

59. assistant (2026-06-18T10:28:06.973Z)

Now let me verify HEAD is still my branch with only my changes vs origin/master before committing (shared-checkout safety).

60. user (2026-06-18T10:28:09.716Z)

BRANCH:
refactor/db-execute-write-helper
HEAD:
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router (#228)
COMMITTS AHEAD OF origin/master:

STATUS:

M backend/infrastructure/database.py
M tests/test_database.py

61. assistant (2026-06-18T10:28:13.310Z)

HEAD == origin/master (no sibling commits), branch correct, only my two files modified. Stage explicit paths and commit.

62. user (2026-06-18T10:28:14.650Z)

```
backend/infrastructure/database.py | 200 ++++++-----  
tests/test_database.py           | 98 ++++++  
2 files changed, 177 insertions(+), 121 deletions(-)
```

63. user (2026-06-18T10:28:24.397Z)

```
[refactor/db-execute-write-helper f7b8a78] refactor(db): route bool-returning single-statement  
writers through _execute_write  
2 files changed, 177 insertions(+), 121 deletions(-)  
---  
f7b8a78 refactor(db): route bool-returning single-statement writers through _execute_write
```

64. assistant (2026-06-18T10:28:26.442Z)

Now push and open the PR.

65. user (2026-06-18T10:28:29.804Z)

```
remote:  
remote: Create a pull request for 'refactor/db-execute-write-helper' on GitHub by visiting:  
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/db-  
execute-write-helper  
remote:  
branch 'refactor/db-execute-write-helper' set up to track 'origin/refactor/db-execute-write-  
helper'.  
To https://github.com/avidullu/badminton-highlight-indexer.git  
* [new branch]      refactor/db-execute-write-helper -> refactor/db-execute-write-helper
```

66. user (2026-06-18T10:28:46.121Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/232>

67. assistant (2026-06-18T10:28:49.589Z)

PR opened: <https://github.com/avidullu/badminton-highlight-indexer/pull/232>

Let me capture the final diffstat for the structured report.

68. user (2026-06-18T10:28:51.243Z)

```
backend/infrastructure/database.py | 200 ++++++-----  
tests/test_database.py           | 98 ++++++  
2 files changed, 177 insertions(+), 121 deletions(-)
```

69. assistant (2026-06-18T10:28:53.121Z)

All four gates green and the PR is open. Returning the structured result.

70. user (2026-06-18T10:29:10.056Z)

Structured output provided successfully

71. assistant (2026-06-18T10:29:13.739Z)

Done. The refactor is complete and PR #232 is open.

Summary: Introduced a private `\_execute\_write(self, sql, params, error\_msg) -> bool` helper in `backend/infrastructure/database.py` and routed the 11 bool-returning single-statement writers through it, preserving each method's exact log message verbatim. Characterization tests were written first (6 write-failure tests pinning the exact per-method log text), confirmed passing on unchanged code, then re-verified passing after the dedup. All four gates green (863 passed/7 skipped, ruff clean, mypy clean, coverage 81.19% ? no regression vs ~80.9% base).